Yapısal Geçmiş Lab ve Uygulamalar

YAPISAL PROGRAMLAMA LAB-1

Soru:

1. N ve M terimli P1 ve P2 polinomları üstler ilk satırda, katsayılar ikinci satırda tutulmak üzere sırasıyla 2*N ve 2*M boyutunda iki matriste tutulmaktadır. Bu iki polinomun çarpımı sonucu oluşacak polinomu hesaplayarak dizi formunda tutan ve ekrana çarpım polinomunun 0'dan farklı terimlerinin ust ve katsayılarını yazan programı C dilinde kodlayınız.

Örn:

p1=2x^2+5x^1 p2=4x^2+2x^1 carpim=8x^4+24x^3+10x^2

P1[][]:

2	1
2	5

P2[][]:

0	0	1	2	8
		0	4	

Carpim[]:

2	1
4	2

Program Hakkında:

- Programda öncelikle terim sayısı kullanıcıdan istenilmelidir
- Terim sayısına bağlı olarak üs ve katsayı değerleri alınmalıdır.
- Çarpım sonucu oluşan polinomun kat sayıları sonuc dizisine aktarılmalıdır.
- Aktarma işleminde pointer kullanılmalıdır.
- Sonuç dizisini ekrana yazdırırken yine pointer kullanılmalıdır
- Programda fonksiyon kullanılmayacaktır, tüm kod main'de olacaktır.
- Kodunuzu ÖğrenciNumaranız.c (Örnek 19011001.c) şeklinde yükleyiniz. C dışındaki program dosyaları(.cpp, .h) değerlendirilmeyecektir.

Örnek Çıktı:

```
ilk polinomun terim sayisi?
2
ust ve katsayi?
2
2
ust ve katsayi?
1
5
ikinci polinomun terim sayisi?
2
ust ve katsayi?
2
4
ust ve katsayi?
1
2
+((10)*p^2)+((24)*p^3)+((8)*p^4)
--------------------------------
```

Uygulama 1:

Zar toplamı ve olasılık matrisi

Toplam	Olasılık
3	0.0046296..
4	0.0138888..
5	0.0277777..
...	...
18	0.0046296..

Üç adet zar beraber atılıyor ve bu zarların üst yüze gelen rakamları toplanıyor.

- Üç zarın olası her bir toplam sonucunu birinci kolonda, olasılığını da ikinci kolonda tutacağınız bir matrisi dinamik bellek allokasyonu ile tanımlayınız.
- Matrisin birinci kolonunda bulunan üç zar toplamı değerleri tekrar etmemelidir (ilk kolon unique olmalıdır).
- İkinci kolondaki olasılık değerlerini döngü/kontrol/aritmetik operasyonlar vb komutlarından uygun olanlarını kullanarak doldurunuz (elle mat[i,j] = 0.0046296 şeklinde "hard coded" bir atama yapmayınız!).

(Kayıt Silinmiş.)

Yapısal Programlama 2. Lab

$$f(n) = \begin{cases} 3n+1 & : \quad \text{if } n \equiv 1 \pmod 2, \\ n/2 & : \quad \text{if } n \equiv 0 \pmod 2 \end{cases}$$

1) Kullanıcıdan pozitif n tam sayısı okunmalı (5 puan)
2) **calloc** fonksiyonu kullanarak, 2 eleman için hafızada yer ayıran, bir boyutlu sepet dizisi oluşturulmalı (10 puan)
3) n tam sayısına f fonksiyonu 1 değeri elde edilene kadar tekrarlı olarak uygulanmalı ve elde edilen her değer sepet dizisine yazılmalıdır. Sepet dizisi dolduğunda ise dizinin boyutu **realloc** fonksiyonu kullanılarak 2 katına çıkarılmalıdır. (70 puan)
4) Sepet dizisi ve hafızada kapladığı alan ekrana yazdırılmalıdır. (5 puan)
5) Sepet dizisi **free** edilmelidir. (10 puan)

```
Not: ptr = realloc(ptr, newSize);
     Mod: bölümünden kalan(% ile ifade edilir)
```

Örnek:
N = 13 başlangıç değeri için aşağıdaki dizi elde edilmelidir.Sepet =
13 40 20 10 5 16 8 4 2 1

0. Adım Sepet = [ ]	eleman sayisi = 0 boyut = 2
1. Adım Sepet = [13]	eleman sayisi = 1 boyut = 2
2. Adım Sepet = [13, 40]	eleman sayisi = 2 boyut = 4
3. Adım Sepet = [13, 40, 20]	eleman sayisi = 3 boyut = 4
4. Adım Sepet = [13, 40, 20, 10]	eleman sayisi = 4 boyut = 8
5. Adım Sepet = [13, 40, 20, 10, 5]	eleman sayisi = 5 boyut = 8
6. Adım Sepet = [13, 40, 20, 10, 5, 16]	eleman sayisi = 6 boyut = 8
7. Adım Sepet = [13, 40, 20, 10, 5, 16, 8]	eleman sayisi = 7 boyut = 8
8. Adım Sepet = [13, 40, 20, 10, 5, 16, 8, 4]	eleman sayisi = 8 boyut = 16
9. Adım Sepet = [13, 40, 20, 10, 5, 16, 8, 4, 2]	eleman sayisi = 9 boyut = 16
10.Adım Sepet = [13, 40, 20, 10, 5, 16, 8, 4, 2, 1]	eleman sayisi = 10 boyut = 16

1 sayısına ulaşıldığı için iterasyon durmuştur.
N = 13 Örnek program çıktısı:

```
N = 13
Dizi elemanlari: 13 40 20 10 5 16 8 4 2 1
Dizinin Eleman Sayisi = 10
Dizinin Hafizada Kapladigi Alan = 16
```

N = 7 Örnek program çıktısı:

```
N = 7
Dizi elemanlari: 7 22 11 34 17 52 26 13 40 20 10 5 16 8 4 2 1
Dizinin Eleman Sayisi = 17
Dizinin Hafizada Kapladigi Alan = 32

--------------------------------
```

Süre: 60dk

LAB 3

- Her bir düğümde v1 ve v2 değerlerini tutacak 2 adet bağlı liste (linked list) yapısı oluşturmanız,
- Bağlı listeye sondan eleman eklemek için bir fonksiyon yazmanız,
- İki bağlı listeyi verilen fonksiyonun şartına göre v1 veya v2 değerlerine göre karşılaştırıp yeni bir bağlı liste döndüren fonksiyonu yazmanız,
- Ayrı ayrı v1 ve v2 değerlerine göre karşılaştırma yapıp node döndüren fonksiyonları yazmanız,
    - 1. Fonksiyonda iki bağlı listenin elemanları karşılaştırılırken aynı v1 değerine sahip iki düğüm bulunuyorsa 1, bulunmuyorsa 0 döndürecek.
    - 2. Fonksiyonda iki bağlı listenin elemanları karşılaştırılırken aynı v2 değerine sahip iki düğüm bulunuyorsa 1, bulunmuyorsa 0 döndürecek.
- V1 ve v2 değerlerine göre karşılaştırma yapıldıktan sonra oluşan bağlı liste ekrana yazdırılması,
- İstenilen şartlara göre elde edilen linkli listelerin print fonksiyonu ile yazdırılması beklenmektedir.

Beklenmektedir.

Aşağıda lab 3 de istenen fonksiyon imzaları ve Struct yapısı verilmiştir.

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int v1;
    int v2;
    struct Node* next;
} Node;
void printList(Node* n); // Verilen Linkli listeyi yazdırır. 10p
void insert(struct Node** head_ref, int v1, int v2); // Verilen değerlere sahip Node'u linkli
listenin sonuna ekler. 10p
Node* intersection(Node* start1, Node* start2, int (*reduce_f)(Node*, Node* ) ); //v1 ve v2 değerine
göre sıralı iki linkli listenin elemanlarının
// verilen fonksiyon şartına göre birleşimini bulur (v1 değeri aynı ise v2 değerini her zaman
1.bağlı listeden alır, v2 değeri aynı ise v1 değerini her zaman 1. Bağlı listeden alır), v1 veya v2
değerleri üzerinde keşim varsa yeni bağlı listeye eleman olarak ekler (yeni bir linkli liste olarak
döndürür). 60p
int compare_by_v1(Node* Node1, Node* Node2); //iki düğümün v1 değerleri aynı ise 1 değilse 0
döndürür. 10p
int compare_by_v2(Node* Node1, Node* Node2); //iki düğümün  v2 değerleri aynı ise 1 değilse 0
döndürür. 10p
```

Örnek:

Girdi:
Linked list 1: v1=1, v2=10 → v1=2, v2=20→v1=3, v2=5
Linked list 2: v1=2, v2=5→v1=1, v2=25 →v1=4, v2=20

Çıktı:
v1'e göre karşılaştırma sonucu : v1=1, v2= 10→ v1=2, v2 =20
v2'ye göre karşılaştırma sonucu : v1=2, v2=20 → v1=3, v2=5

Süre: 60dk

Uygulama-4 Dosya ve Veritabanı Yönetimi

Bir üniversitenin personel veritabanını yöneten bir C programı geliştirilmesi istenmiştir. Program, personel bilgilerini içeren bir dosyadan okuma, bu bilgileri işleme, sıralama ve güncellenmiş verileri yeni dosyalara yazma işlevlerini gerçekleştirecek. Her personelin sırasıyla bir ID'si, ismi ve çalıştığı departman bilgisi bulunacak.

İstenenler:

- Program, **"Dataset.txt"** isimli bir dosyadan virgülle ayrılmış personel bilgilerini parse ederek okumalıdır. Örnek personel bilgisi aşağıdaki gibidir:

  4,Ali,BLM
  200,Ahmet,ELM
  21,Mehmet,KOM
  2,Ayse,EHM

- Okunan her satır, bir "**data**" yapısına dönüştürülecek ve bunlar bir linked list içinde saklanacaktır.
- **data** yapısı ID, Name, department ve sonraki data objesine bir pointer (next) içermelidir.
- Okunan ve işlenen veriler linked list üzerinden sırasıyla personel ID'lerine ait veri satırlarını orijinal, azalan ve artan sıralamayla ekrana yazdırılmalıdır.
- Azalan sırayla sıralanmış liste **"New_dataset_descending.txt"** isimli dosyaya yazılmalı
- Artan sırayla sıralanmış liste **"New_dataset_ascending.txt"** isimli dosyaya yazılmalı.
- Her iki dosya da orijinal girdi dosyasıyla aynı formata (virgülle ayrılmış) sahip olmalıdır.

Örnek çıktı:

```
4,Ali,BLM
200,Ahmet,ELM
21,Mehmet,KOM
2,Ayse,EHM
ID: 4, Name: Ali, Department: BLM
ID: 200, Name: Ahmet, Department: ELM
ID: 21, Name: Mehmet, Department: KOM
ID: 2, Name: Ayse, Department: EHM
Descending
ID: 200, Name: Ahmet, Department: ELM
ID: 21, Name: Mehmet, Department: KOM
ID: 4, Name: Ali, Department: BLM
ID: 2, Name: Ayse, Department: EHM
Ascending
ID: 2, Name: Ayse, Department: EHM
ID: 4, Name: Ali, Department: BLM
ID: 21, Name: Mehmet, Department: KOM
ID: 200, Name: Ahmet, Department: ELM
```

(Kayıt silinmiş)

LAB - 4

Soru:
Verilen text dosyasının her satırı havaalanından veya havaalanına yapılacak olan bir uçuşa ait şu bilgileri sırasıyla içermektedir: Benzersiz bir uçuş kodu(4 haneli sayı), kalkış şehri, varış şehri, pistte bulunacağı saat, iç ya da dış hat olma bilgisi(0: İç hatlar, 1: Dış Hatlar) ve kullanacağı pist numarası(pozitif tam sayı).

Havaalanındaki pistlerin kullanım durumuna ilişkin her pist için birer text dosyasına pistin kullanılacağı seferlerin uçuş bilgilerini yazan bir algoritmayı C dilinde gerçekleyiniz. Programınız aşağıdaki özellikleri bulundurmalıdır:

1. Uçuşları ifade eden bir struct yapısı kurunuz.
2. Dosyadaki uçuşları uçuş koduna göre küçükten büyüğe sıralı olarak bir linkli listeye kaydediniz.
3. Sıralı listeyi ekranda gösteriniz.
4. Her bir pist için pistin adını taşıyan birer dosya oluşturarak (Örn: pist1.txt) pisti kullanacak uçuşların bilgilerini bu dosyaya kaydediniz.

Örnek girdi dosyası içeriği:

3001 ISTANBUL ANKARA 13:00 0 1
2013 ISTANBUL LONDON 12:00 1 8
1015 LONDON ISTANBUL 20:00 1 2
1012 SIVAS ISTANBUL 14:00 0 1

Çıktı 1: Ekrana yazdırılan (Linked List):

1012 SIVAS ISTANBUL 14:00 0 1
1015 LONDON ISTANBUL 20:00 1 2
2013 ISTANBUL LONDON 12:00 1 8
3001 ISTANBUL ANKARA 13:00 0 1

Çıktı 2 : Oluşturulan Dosyalar ve içerikleri:

Dosya-1: pist1.txt :

1012 SIVAS ISTANBUL 14:00 0 1
3001 ISTANBUL ANKARA 13:00 0 1

Dosya-2: pist2.txt :

1015 LONDON ISTANBUL 20:00 1 2

Dosya-3: pist8.txt:

2013 ISTANBUL LONDON 12:00 1 8

Süre: 45 dakika (40dk+5 dk yükleme)

... good luck